Open in app ↗ Sign up Sign in

Medium Search

Make Android · Follow publication

Featured

Aosp + Android + Automotive + Android Framework + Android App Development +

Build AOSP Android 16 Automotive OS for Raspberry Pi 5 and flash it.

This article outlines the process of building Android 16 Automotive OS for the Raspberry Pi 5, including the setup of the AOSP build environment, creating a flashable SD card image, flashing the device, and highlighting the issues, challenges, and key takeaways encountered along the way.

11 min read · Sep 7, 2025

 Narendra Harny Follow

Listen Share

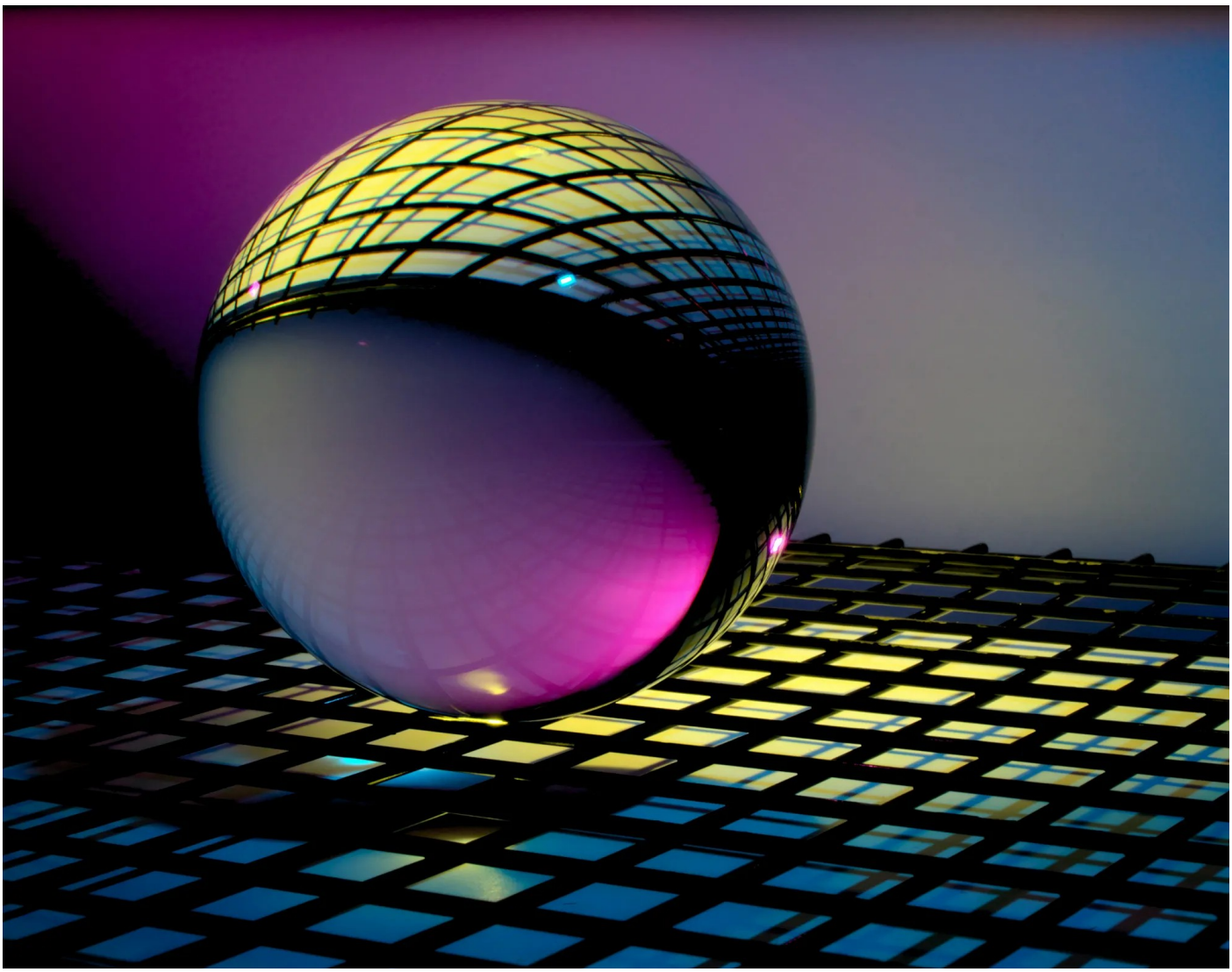


Photo by Michael Dziedzic on Unsplash

The Raspberry Pi 5 serves as a robust platform for exploring Android Automotive and IoT innovations. This guide takes you through the process of building AOSP Android 16 for the Raspberry Pi 5, integrating it smoothly with the hardware, and discovering new possibilities beyond traditional use cases. Whether you're an Android enthusiast or a developer eager to experiment with the latest technology, this article offers a clear and practical roadmap to get you started.

Let's dive in!

Devices Configuration and Hardware Setup

Build PC configurations for me:

> *CPU: AMD Ryzen 9 9950TM-16 Core*
>
> *OS: Ubuntu 22.04.5 LTS*
>
> *RAM: 64 GB*
>
> *SSD: 500 GB*

Hardware List:

> **SD Card 64 GB + USB card reader**

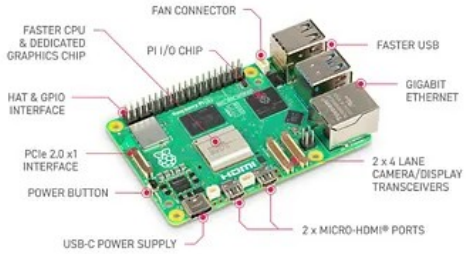
> **Power supply Adapter 5.1 V**
>
> **Rasberypi 5 Modle B Board without computation modle.**
>
> **UCB to Type C Cable**
>
> **Mini HDMi to HDMI cable**











Tools on Build Machine:

> **Git, Repo**
>
> **OpenJDK-17-jdk**
>
> **imager_1.8.5_amd64 or Balena-etcher**

The setup described above is required for this task. However, if you have alternative configurations that work reliably, those are equally suitable to get started.

OpenJDK-17-jdk

```
sudo apt-get update
sudo apt-get install openjdk-17-jdk
java -version
```

Git

```
sudo apt-get install git
git config - global user.name "Your Name"
git config - global user.email "youremail@domain.com"
```

Download Android 16 AOSP Source for Raspberry Pi 5

Prepare build environment (Ubuntu 22.04 LTS recommended)

```
sudo apt-get update

sudo apt-get install git-core gnupg flex bison gperf build-essential \
    zip curl zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 lib32ncurses5-dev \
    x11proto-core-dev libx11-dev lib32z-dev ccache libgl1-mesa-dev libxml2-utils \
    xsltproc unzip fontconfig
```

The packages installed using `sudo apt-get install` . These are essential for setting up the Android Open Source Project (AOSP) build environment. `git-core` is used for source code management and repository synchronisation, while `gnupg` provides secure authentication and verification of downloads. Tools such as `flex` , `bison` , and `gperf` are required for parsing, code generation, and configuration processing. The `build-essential` package installs core compilation tools, including `gcc` , `g++` , and `make` . Utilities like `zip` , `unzip` , and `curl` handle packaging, extraction, and downloading of files. Libraries such as `zlib1g-dev` and `lib32z-dev` Provide the compression support required by Android build tools. On 64-bit hosts, `gcc-multilib` , `g++-multilib` , and `libc6-dev-i386` enable 32-bit compatibility, while `lib32ncurses5-dev` providing terminal handling for 32-bit builds. X11 libraries, including `x11proto-core-dev` and `libx11-dev` , are necessary for building graphical components. `ccache` improves build performance by caching compilation outputs, and `libgl1-mesa-dev` supplies OpenGL libraries for rendering. `libxml2-utils` and `xsltproc` enable XML and XSLT processing for build scripts, while `fontconfig` ensuring proper font handling and configuration. Together, these tools provide the compilers, libraries, and system dependencies required to build Android from source successfully.

Installing these packages may take some time. Please allow the process to complete, and ensure all packages are installed successfully. If any errors occur, monitor the installation logs to identify and resolve them.

Install the repo tool:

If you have built AOSP before, you may have already completed this step and have `repo` installed and set up on your PC.

You can skip the “Install the repo tool” step if it was already installed during a previous AOSP setup. In that case, you will see the following message:

```
... A new version of repo (2.54) is available.
... You should upgrade soon:
```

Then, specifically repeat *“Install the repo tool:”*. After repeating *“Install the repo tool:”*, continue with the next step.

```
mkdir -p ~/bin
curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo
chmod a+x ~/bin/repo
export PATH=~/bin:$PATH
```

Create Workspace:

```
mkdir Android16
```

```
cd Android16
```

Initialise repo:

I am using the AOSP Android 16.0.0_r1 branch, which is the initial public release of Android 16. This branch provides a stable and well-documented baseline, making it ideal for development, experimentation, and building custom images for the Raspberry Pi 5. Additionally, the local manifest repository https://github.com/raspberry-vanilla/android_local_manifest?tab=readme-ov-file mentions updates for AOSP Android 16.0.0_r1, which is why we will use it for our setup.

```
repo init -u https://android.googlesource.com/platform/manifest -b android-16.0.0_r1
```

Output:

```
repo: reusing existing repo client checkout in ../..WorkSpace/AOSP/Android16

repo has been initialized in ../../WorkSpace/AOSP/Android16
```

Add the Raspberry Pi Local Manifest

You can find the local manifest for Raspberry Pi here:
https://github.com/raspberry-vanilla/android_local_manifest

Required tools:

```
sudo apt-get install dosfstools e2fsprogs fdisk kpartx mtools rsync
```

Adding Local Manifest:

```
mkdir -p .repo/local_manifests
git clone https://github.com/raspberry-vanilla/android_local_manifest.git .repo/local_manifests
```

|| OR ||

```
curl -o .repo/local_manifests/manifest_brcm_rpi.xml -L https://raw.githubusercontent.com/raspberry-vanilla/andro
```

|| OR ||

> *Or optionally, you can reduce download size by creating a shallow clone and removing unneeded projects:*

```
repo init -u https://android.googlesource.com/platform/manifest -b android-16.0.0_r1 --depth=1
curl -o .repo/local_manifests/manifest_brcm_rpi.xml -L https://raw.githubusercontent.com/raspberry-vanilla/andro
curl -o .repo/local_manifests/remove_projects.xml -L https://raw.githubusercontent.com/raspberry-vanilla/androi
```

This manifest automatically pulls in `device/raspberry/*`, `kernel`, and vendor repos for RPi5.

I ran the first command, and the output appears as follows:

```
Cloning into '.repo/local_manifests'...
remote: Enumerating objects: 329, done.
remote: Counting objects: 100% (86/86), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 329 (delta 74), reused 60 (delta 60), pack-reused 243 (from 2)
Receiving objects: 100% (329/329), 45.64 KiB | 730.00 KiB/s, done.
Resolving deltas: 100% (166/166), done.
```

Note: The manifest in this repository https://github.com/raspberry-vanilla/android_local_manifest?tab=readme-ov-file has been updated for Android 16.0.0_r1. Ensure that you build AOSP for the corresponding branch if you plan to flash it on a Raspberry Pi.

Sync source:

```
repo sync -j$(nproc --all)

or

repo sync -j[n] == repo sync -j12
```

In `-j<n>`, where `n` is the number of cores available on your build machine. It's recommended to leave 4 cores for the system and use the remaining cores for the build. To ensure better performance and avoid repo sync failures, explicitly specify the number of cores when running `repo sync` using `repo sync -j<n>`.

Here's an **optimised `repo sync` command**

```
repo sync -c -j12 --force-sync --no-clone-bundle --no-tags
```

> ***`-c`*** *→ Sync only the current branch (saves space/time vs all branches).*
>
> ***`-j12`*** *→ Run 12 parallel jobs (matches your cores).*
>
> ***`--force-sync`*** *→ Cleans up and resyncs repos if something gets corrupted.*
>
> ***`--no-clone-bundle`*** *→ Disables using prebuilt Git bundle (smaller initial download, fewer failures).*
>
> ***`--no-tags`*** *→ Skips fetching Git tags (faster & smaller checkout, not needed for builds).*

I have not used any of these options. Try adding them and observing the result.

Repo sync is completed!

```
Receiving objects: 100% (2794/2794), 1.88 MiB | 112.97 MiB/s, done.
Resolving deltas: 100% (14/14), done.
Receiving objects: 100% (2868/2868), 902.98 KiB | 82.09 MiB/s, done.eiving objects:   0% (1/2868)
Resolving deltas: 100% (19/19), done.
Receiving objects: 100% (3262/3262), 1.15 MiB | 73.92 MiB/s, done.
Resolving deltas: 100% (211/211), done.
Receiving objects: 100% (3876/3876), 1.51 MiB | 73.51 MiB/s, done.eceiving objects:   0% (1/3876)
Resolving deltas: 100% (114/114), done.
Receiving objects: 100% (13073/13073), 3.18 MiB | 69.29 MiB/s, done.eiving objects:   0% (1/13073)
Resolving deltas: 100% (7579/7579), done.
Receiving objects: 100% (22454/22454), 3.42 MiB | 68.66 MiB/s, done.eiving objects:   0% (1/22454)
Resolving deltas: 100% (11860/11860), done.
Receiving objects: 100% (3418/3418), 1.02 MiB | 61.15 MiB/s, done.eceiving objects:   0% (1/3418)
Resolving deltas: 100% (128/128), done.
Receiving objects: 100% (5468/5468), 7.51 MiB | 124.00 MiB/s, done.ceiving objects:   0% (1/5468)
Resolving deltas: 100% (3819/3819), done.
Receiving objects: 100% (3611/3611), 1.10 MiB | 86.72 MiB/s, done.eceiving objects:   0% (1/3611)
Resolving deltas: 100% (591/591), done.
Receiving objects: 100% (4520/4520), 1.49 MiB | 76.23 MiB/s, done.eceiving objects:  82% (103138/125777), 45.
Resolving deltas: 100% (550/550), done.
Receiving objects: 100% (125777/125777), 134.71 MiB | 99.17 MiB/s, done.ng objects:  87% (109426/125777), 45.
Resolving deltas: 100% (109906/109906), done.8 platform/cts @ ctsResolving deltas:  95% (104411/109906)
Receiving objects: 100% (2830/2830), 1.84 MiB | 104.83 MiB/s, done.ceiving objects:   0% (1/2830)
Resolving deltas: 100% (97/97), done.
Syncing: 25% (250/982) 16:51 | 12 jobs | 16:49 platform/cts @ cts
```

> *Repo sync completed after 2 hrs 13 min!!*

Set up Android build environment:

```
source build/envsetup.sh
```

Check for lunch options:

```
lunch
```

Output Lunch Options:

As observed when building Android 15, the lunch options are similarly available in Android 16.

```
aosp_rpi4_car-trunk_staging-userdebug
aosp_rpi4-trunk_staging-userdebug
aosp_rpi4_tv-trunk_staging-userdebug

---> aosp_rpi5_car-trunk_staging-userdebug

aosp_rpi5-trunk_staging-userdebug
aosp_rpi5_tv-trunk_staging-userdebug
```

While building Android 15, the lunch options were not visible when I ran the `lunch` command. Refer to the **"Build Issues"** section below for details.

Get Narendra Harny's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Lunch option Android 16 for Rpi 5 is →

```
lunch aosp_rpi5_car-bp2a-userdebug
```

Output:

```
============================================
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=16
TARGET_PRODUCT=aosp_rpi5_car
TARGET_BUILD_VARIANT=userdebug
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a76
HOST_OS=linux
HOST_OS_EXTRA=Linux-6.8.0-64-generic-x86_64-Ubuntu-22.04.5-LTS
HOST_CROSS_OS=windows
BUILD_ID=BP2A.250605.031.A2
OUT_DIR=out
SOONG_ONLY=false
============================================

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  Wondering whether to use user, userdebug or eng?

  user        The builds that ship to users. Reduced debugability.
  userdebug   High fidelity to user builds but with some debugging options
              enabled. Best suited for performance testing or day-to-day use
              with debugging enabled.
  eng         More debugging options enabled and faster build times, but
              runtime performance tradeoffs. Best suited for day-to-day
              local development when not doing performance testing.
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

Build AOSP-15 System Images

```
make bootimage systemimage vendorimage -j$(nproc)

nproc can be replaced accordingly, 12 for me.

make bootimage systemimage vendorimage -j12
```

> *The build has started! Time to grab a coffee.*

```
============================================
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=16
TARGET_PRODUCT=aosp_rpi5_car
TARGET_BUILD_VARIANT=userdebug
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a76
HOST_OS=linux
HOST_OS_EXTRA=Linux-6.8.0-64-generic-x86_64-Ubuntu-22.04.5-LTS
HOST_CROSS_OS=windows
BUILD_ID=BP2A.250605.031.A2
OUT_DIR=out
SOONG_ONLY=false
============================================
[ 99% 132/133] analyzing Android.bp files and generating ninja file at out/soong/build.aosp_rpi5_car.ninja
    0:32 analyzing Android.bp files and generating ninja file at out/soong/build.aosp_rpi5_car.ninja
```

Build Completed!!

```
[ 99% 150357/150366] //external/icu/android_icu4j:stable.i18n.module.platform.api.stubs.source metalava merg
external/icu/android_icu4j/src/main/java/android/icu/text/Transliterator.java:455: warning: Public class and
ext.StringTransform (ErrorWhenNew) [HiddenSuperclass]
external/icu/android_icu4j/src/main/java/android/icu/text/Transliterator.java:455: warning: Public class and
ext.Transform (ErrorWhenNew) [HiddenSuperclass]
external/icu/android_icu4j/src/main/java/android/icu/text/UCharacterIterator.java:30: warning: Public class
d.icu.text.UForwardCharacterIterator (ErrorWhenNew) [HiddenSuperclass]
[ 99% 150365/150366] //frameworks/base/api:system-api-stubs-docs-non-updatable metalava merged (priority: 0)
frameworks/base/telecomm/java/android/telecom/TelecomManager.java:2853: warning: When a method overload is n
ext.createPackageContextAsUser() and re-obtain the relevant Manager, and no new API should be added (ErrorWh
metalava wrote updated baseline to /media/narendra147h/UbuntuDrive/WorkSpace/AOSP/Android16/out/soong/.temp/
[100% 150366/150366] //frameworks/base/api:system-api-stubs-docs-non-updatable check current API (priority:

#### build completed successfully (01:07:51 (hh:mm:ss)) ####
```

This will generate files such as `out/target/product/rpi5/system.img`, `boot.img`, and others. With the Raspberry Pi build now complete, we can proceed to prepare the flashing image.

Let's check that!!

```
cd out/target/product/rpi5/

ls -la
```

```
      4096 Sep  7 11:47 .
      4096 Sep  7 10:26 ..
   2132029 Sep  7 10:41 all_modules.txt
        14 Sep  7 11:46 android-info.txt
      4096 Sep  7 11:46 apex
 134217728 Sep  7 11:46 boot.img
        82 Sep  7 10:26 build_fingerprint.txt
        53 Sep  7 10:41 build_thumbprint.txt
    169381 Sep  7 10:41 clean_steps.mk
        37 Sep  7 10:41 .copied_headers_list
      4096 Sep  7 11:46 debug_ramdisk
     12288 Sep  7 11:46 dexpreopt_config
      4096 Sep  7 11:46 fake_packages
      4096 Sep  7 11:38 gen
   3379189 Sep  7 10:41 .installable_files.previous
    539076 Sep  7 11:46 installed-files.json
       286 Sep  7 11:46 installed-files-ramdisk.json
        62 Sep  7 11:46 installed-files-ramdisk.txt
      3139 Sep  7 11:46 installed-files-root.json
       674 Sep  7 11:46 installed-files-root.txt
    186925 Sep  7 11:46 installed-files.txt
     89794 Sep  7 11:46 installed-files-vendor.json
     28145 Sep  7 11:46 installed-files-vendor.txt
  35371520 Sep  7 10:48 kernel
      4096 Sep  7 11:46 obj
        49 Sep  7 10:42 previous_build_config.mk
         0 Sep  7 10:42 previous_use_partial_compile.txt
     16619 Sep  7 11:43 product_packages.txt
      4096 Sep  7 11:46 ramdisk
   1831354 Sep  7 11:46 ramdisk.img
      4096 Sep  7 11:46 root
      4096 Sep  7 11:46 rpiboot
      4096 Sep  7 11:48 symbols
      4096 Sep  7 11:46 system
2684354560 Sep  7 11:47 system.img
      4096 Sep  7 11:46 vendor
 268435456 Sep  7 11:46 vendor.img
```

The `RaspberryVanillaAOSP16-20250907-rpi5_car.img` has been generated, which is our final target. Let's proceed to build the final image.

Build `RaspberryVanillaAOSP16-20250907-rpi5_car.img`

```
croot # go back to root dir
```

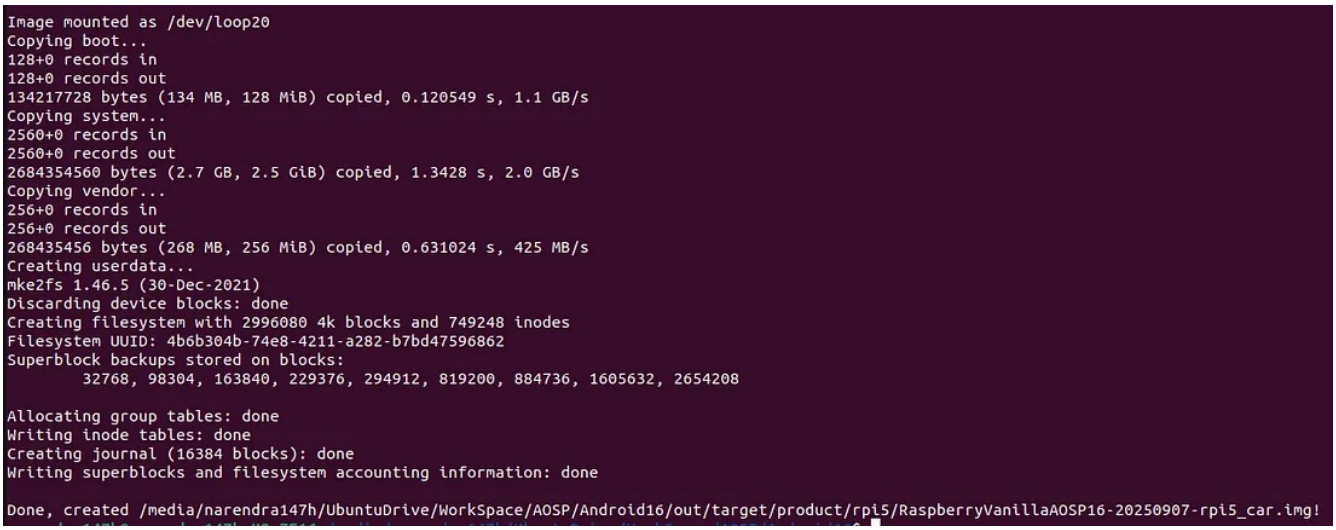
In the Root dir of AOPS "Android16/", you will see the following files:

```
  25 Sep  7 00:58 rpi4-mkimg.sh -> device/brcm/rpi4/mkimg.sh
  25 Sep  7 00:58 rpi4-wrimg.sh -> device/brcm/rpi4/wrimg.sh
  25 Sep  7 00:58 rpi5-mkimg.sh -> device/brcm/rpi5/mkimg.sh
  25 Sep  7 00:58 rpi5-wrimg.sh -> device/brcm/rpi5/wrimg.sh
4096 Sep  7 08:42 sdk
```

Run:

```
./rpi5-mkimg.sh
```

Success Output:



Note: The final part of the output — `"Done, created out/target/product/rpi5/RaspberryVanillaAOSP16-20250907-rpi5_car.img!"` — should appear when the build completes successfully. If any errors occur, this message will not be displayed, and an error will be shown instead.

Verify Images

Once the build is complete, verify that the following files are present: `RaspberryVanillaAOSP16-20250907-rpi5_car.img`, `system.img`, `vendor.img`, and `boot.img`.

```
cd ../../android-aosp/out/target/product/rpi5/
ls -la
```

Output contains data `RaspberryVanillaAOSP16-20250907-rpi5_car.img` generated with a size of 15360000000.

```
      4096 Sep  7 12:03 .
      4096 Sep  7 10:26 ..
   2132029 Sep  7 10:41 all_modules.txt
        14 Sep  7 11:46 android-info.txt
      4096 Sep  7 11:46 apex
 134217728 Sep  7 11:46 boot.img
        82 Sep  7 10:26 build_fingerprint.txt
        53 Sep  7 10:41 build_thumbprint.txt
    169381 Sep  7 10:41 clean_steps.mk
        37 Sep  7 10:41 .copied_headers_list
      4096 Sep  7 11:46 debug_ramdisk
     12288 Sep  7 11:46 dexpreopt_config
      4096 Sep  7 11:46 fake_packages
      4096 Sep  7 11:38 gen
   3379189 Sep  7 10:41 .installable_files.previous
    539076 Sep  7 11:46 installed-files.json
       286 Sep  7 11:46 installed-files-ramdisk.json
        62 Sep  7 11:46 installed-files-ramdisk.txt
      3139 Sep  7 11:46 installed-files-root.json
       674 Sep  7 11:46 installed-files-root.txt
    186925 Sep  7 11:46 installed-files.txt
     89794 Sep  7 11:46 installed-files-vendor.json
     28145 Sep  7 11:46 installed-files-vendor.txt
  35371520 Sep  7 10:48 kernel
      4096 Sep  7 11:46 obj
        49 Sep  7 10:42 previous_build_config.mk
         0 Sep  7 10:42 previous_use_partial_compile.txt
     16619 Sep  7 11:43 product_packages.txt
      4096 Sep  7 11:46 ramdisk
   1831354 Sep  7 11:46 ramdisk.img
15360000000 Sep  7 12:03 RaspberryVanillaAOSP16-20250907-rpi5_car.img
      4096 Sep  7 11:46 root
      4096 Sep  7 11:46 rpiboot
      4096 Sep  7 11:48 symbols
      4096 Sep  7 11:46 system
 2684354560 Sep  7 11:47 system.img
      4096 Sep  7 11:46 vendor
  268435456 Sep  7 11:46 vendor.img
```

Prepare SD Card

Once the AOSP system images are built, there are two easy-to-use software options to flash them onto an SD card: Raspberry Pi Imager or Balena Etcher. Both are available for Windows and Linux.

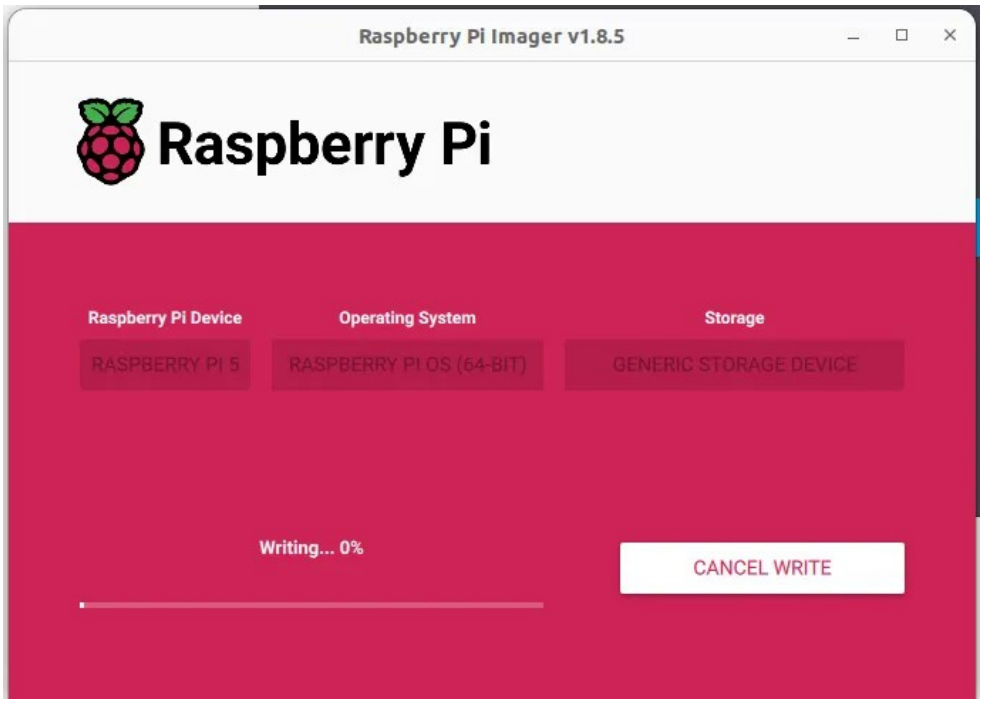
For this guide, I will use Raspberry Pi Imager for specific reasons mentioned in the “Issues” section. However, both tools are excellent and straightforward to use.

Steps to Flash the SD Card:

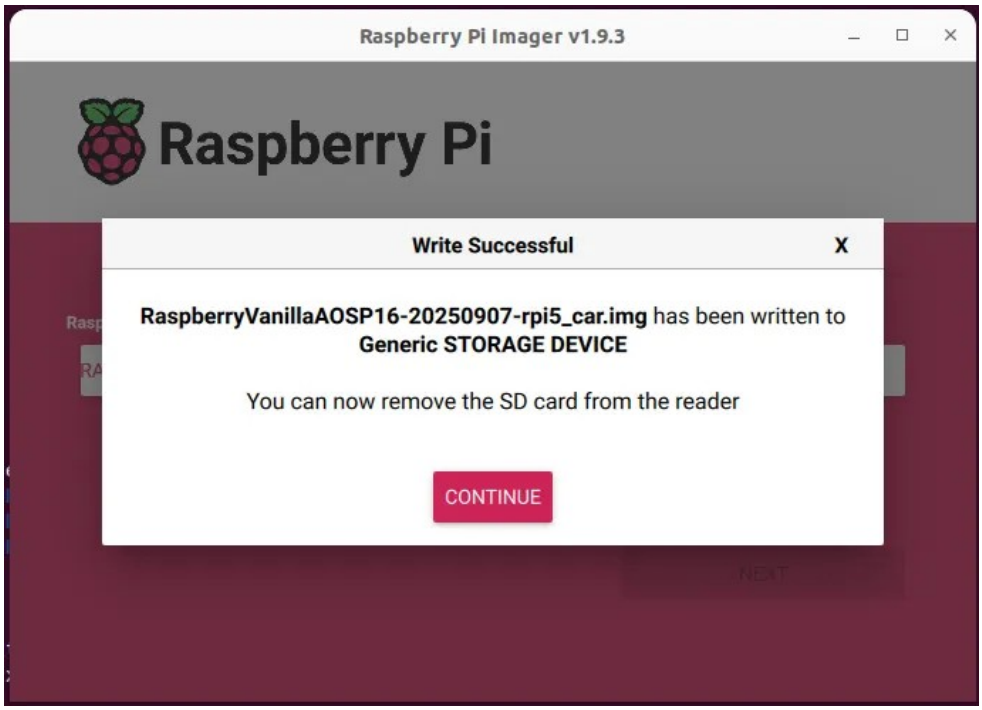
1. Open Raspberry Pi Imager.
2. Select the Raspberry Pi device: **Raspberry Pi 5**.
3. Under Operating System, choose **Custom** and select `RaspberryVanillaAOSP16-20250907-rpi5_car.img`.
4. Select the SD card under **Storage**.

Click “Yes” without changing any settings. If you need to modify any options, feel free to do so.

The tool will first write the image and then verify it; both steps may take some time, so it’s a good opportunity to grab a coffee.

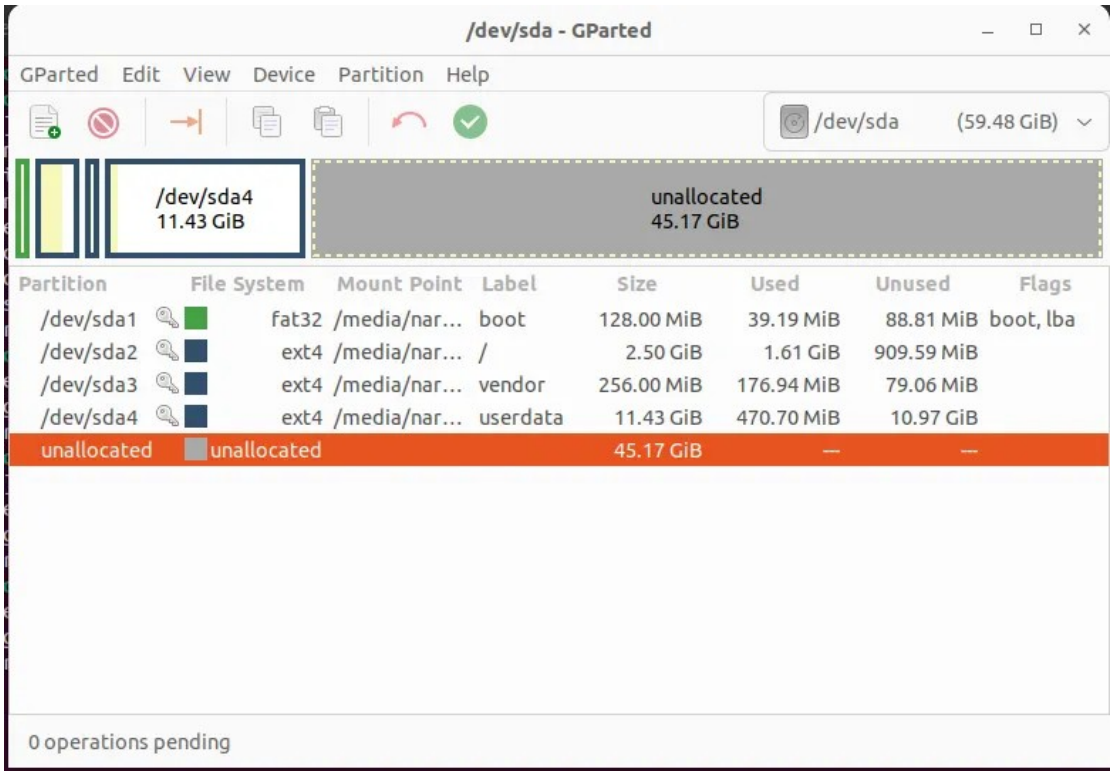

Raspberry Pi Imager v1.8.5
Raspberry Pi
Raspberry Pi Device
Operating System
Storage
RASPBERRY PI 5
RASPBERRY PI OS (64-BIT)
GENERIC STORAGE DEVICE
Writing... 0%
CANCEL WRITE


Raspberry Pi Imager v1.8.5
Raspberry Pi
Raspberry Pi Device
Operating System
Storage
RASPBERRY P...
RASPBERRYVANILLAAOSP15-20250120-...
GENERIC STORAGE DEVICE
Verifying... 7%
CANCEL VERIFY



The `RaspberryVanillaAOSP16-20250907-rpi5_car.img` The SD card is now ready to flash onto the Raspberry Pi 5 board.

After successfully flashing the SD card with — `RaspberryVanillaAOSP16-20250907-rpi5_car.img` Your SD card partitions will appear as shown below.



Here is the complete article to prepare the SD card!

Preparing SD card to Flash Raspberry PI-5 with Android-15

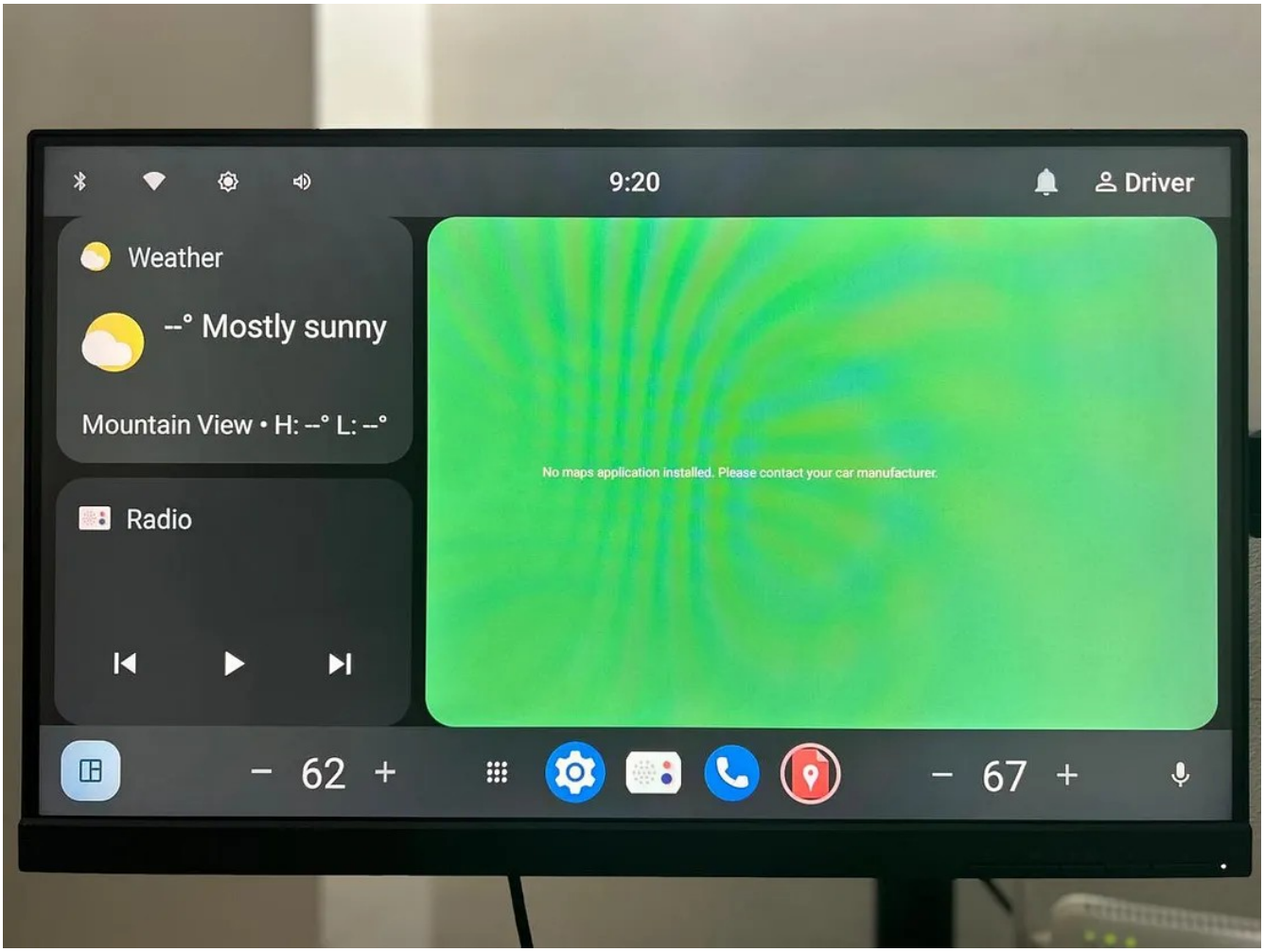
This article explains all the steps for SD Card recovery and preparing it for flashing. These steps are common and not...

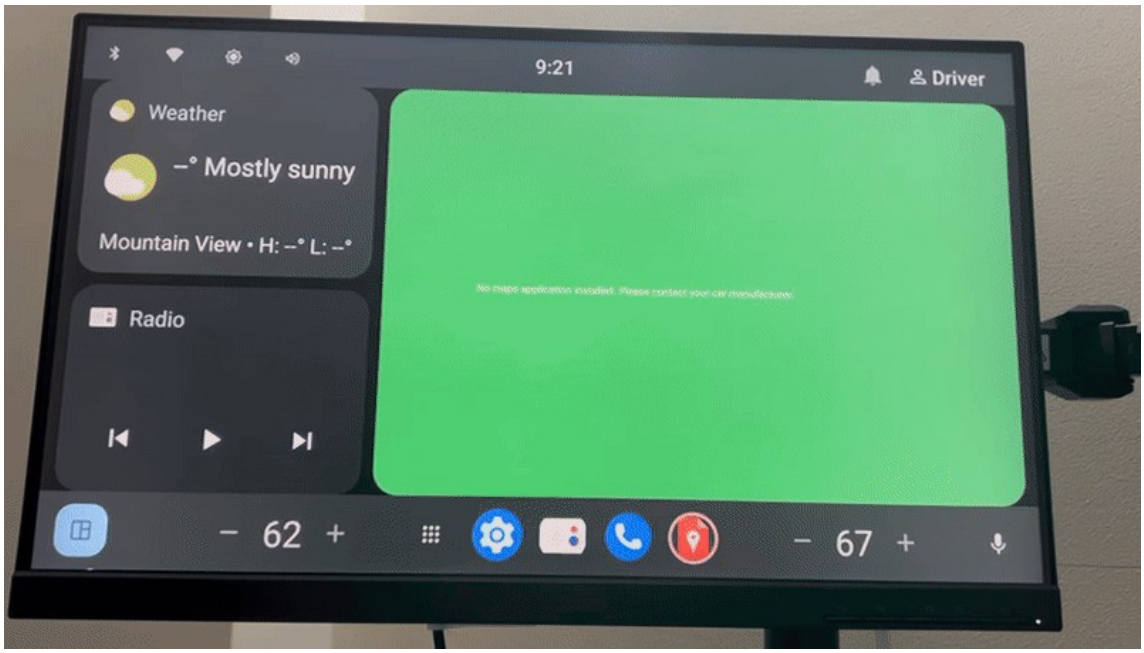
narendra147h.medium.com

Start Raspberry Pi 5 Board with AOSP 16 flashed to SD card

Insert the SD card into the Raspberry Pi 5 board and power it on. The board will boot and flash Android 16. If the power LED blinks, it indicates the Pi is starting up. After a few minutes, the Android logo will appear on the screen.

Finally, I can see Android-16 Automotive OS on Display





Android Automotive + Rpi 5 B

Issue and Key Points Captured

The installation of tools required for the build environment can take a significant amount of time.

Installing `openjdk-17-headless` alone took nearly an hour.

```
sudo apt-get install git-core gnupg flex bison gperf build-essential \
    zip curl zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 lib32ncurses5-dev \
    x11proto-core-dev libx11-dev lib32z-dev ccache libgl1-mesa-dev libxml2-utils \
    xsltproc unzip fontconfig
```

Open-JDK 17 installation error:

```
33% [11 openjdk-17-jdk-headless 25.8 MB/71.3 MB 36%]
Get:12 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 openjdk-17-jdk amd64 17.0.16+8~us1-0ubur
Fetched 72.6 MB in 2h 29min 26s (8,094 B/s)
E: Failed to fetch http://security.ubuntu.com/ubuntu/pool/universe/o/openjdk-17/openjdk-17-jre-headless_17.0.16%
E: Unable to fetch some archives, maybe run apt-get update or try with --fix-missing?
```

Solution:

```
E: Unable to fetch some archives, maybe run apt-get update or try with --fix-missing?
```

Build Issue:

lunch command:

```
lunch

No target specified. See lunch --help
```

lunch — help:

```
The lunch command selects the configuration to use for subsequent
Android builds.

Usage: lunch TARGET_PRODUCT [TARGET_RELEASE [TARGET_BUILD_VARIANT]]

  Choose the product, release and variant to use. If not
  supplied, TARGET_RELEASE will be 'trunk_staging' and
  TARGET_BUILD_VARIANT will be 'eng'


Usage: lunch TARGET_PRODUCT-TARGET_RELEASE-TARGET_BUILD_VARIANT

  Chose the product, release and variant to use. This
  legacy format is maintained for compatibility.


Note that the previous interactive menu and list of hard-coded
list of curated targets has been removed. If you would like the
list of products, release configs for a particular product, or
variants, run list_products list_releases or list_variants
respectively.
```

If you execute lunch and want to see menu options that probably will not show, in that case, you need to have a look at “Android16/device/generic/car/” in AndroidProducts.mk configurations.

```
COMMON_LUNCH_CHOICES := \
    gsi_car_arm64-trunk_staging-userdebug \
    gsi_car_x86_64-trunk_staging-userdebug \
    sdk_car_arm64-trunk_staging-userdebug \
    sdk_car_md_x86_64-trunk_staging-userdebug \
    sdk_car_cw_x86_64-trunk_staging-userdebug \
    sdk_car_x86_64-trunk_staging-userdebug \
```

The above options are for a car emulator and other variants. To find the correct lunch option, look at →

```
cat ./brcm/rpi5/AndroidProducts.mk
```

```
PRODUCT_MAKEFILES := \
    $(LOCAL_DIR)/aosp_rpi5.mk \
    $(LOCAL_DIR)/aosp_rpi5_car.mk \
    $(LOCAL_DIR)/aosp_rpi5_tv.mk

COMMON_LUNCH_CHOICES := \
    aosp_rpi5-trunk_staging-userdebug \

    ---> aosp_rpi5_car-trunk_staging-userdebug \
```

```
aosp_rpi5_tv-trunk_staging-userdebug
```

When you run →

```
lunch aosp_rpi5_car-trunk_staging-userdebug
```

You may see →

```
panic: Missing config trunk_staging.  Trace=[trunk_staging]

goroutine 1 [running]:
main.main()
 build/soong/cmd/release_config/release_config/main.go:71 +0x14c5
In file included from build/make/core/config.mk:419:
In file included from build/make/core/envsetup.mk:51:
build/make/core/release_config.mk:142: error: release-config failed to run.
10:04:49 dumpvars failed with: exit status 1
```

Solution:

```
lunch aosp_rpi5_car-bp2a-userdebug
```

Flashing Issue:

Sometimes, the Raspberry Pi board may fail to boot or display output. In such cases, ensure that the SD card was flashed correctly. If needed, reformat the SD card with the FAT32 file system and flash the image again.

I have noted a few issues encountered while flashing the Raspberry Pi 5 with Android 15. Please refer to the article if you face similar problems.

Build AOSP Android 15 Automotive OS for Raspberry Pi 5 and flash it.

This article will cover building Android-15 Automotive OS, building Raspberry PI5 Image, preparing setup for aosp...

narendra147h.medium.com

> *Thank you for reading!*
>
> *Please feel free to leave your questions and suggestions in the comments. If you found this blog helpful, don't forget to give it a clap! You can also follow me on Medium, connect on LinkedIn, follow on Instagram, or send an email for any further discussion on this topic.*
>
> *Thank you!*

Aosp + Android + Automotive + Android Framework + Android App Development +



Follow

Written by Narendra Harny

558 followers · 57 following

Connect on https://medium.com/make-android | Write On, Android AOSP & Applications | Python | DevOps | Java | C++ | Kotlin | Shell | Linux | Android Auto | IVI



Follow

Published in Make Android

226 followers · Last published Apr 9, 2025

Make Android your writing space into Android, Java, Kotlin, Shell, Linux, Gradle, and Groovy. Uncover the future of technology through our in-depth articles, expert insights, and cutting-edge innovations. Stay on the forefront of Android's evolution with us.